Circuit Breaker Pattern

Definition

A **Circuit Breaker** is a fault-tolerance mechanism that **temporarily stops requests to a failing service** when error rate crosses a threshold.

It prevents repeatedly calling a service that is likely down or overloaded.

Simple Meaning

> If a dependency keeps failing, stop calling it for some time.

Why Needed

Without circuit breaker:

- Thousands of retries hit failing service
- More load on already broken system
- Cascading failures
- Resource waste
- Increased latency

With circuit breaker:

- Protect downstream services
- Fast failure response
- Better system stability
- Prevent chain outages

Upstream vs Downstream

If Service B calls Service A:

- **B = Upstream service**
- **A = Downstream service**

Example:

Checkout Service → Payment Service

- Checkout = Upstream
- Payment = Downstream

Real Example

User places order.

Checkout service calls Payment API.

Payment API starts timing out.

Without circuit breaker:

- Every request waits 10 sec timeout

With circuit breaker:

- Stop calling payment temporarily
- Return fallback response immediately

Circuit Breaker States

1. Closed State (Normal)

Everything healthy.

- Requests flow normally.
- Failures counted.

```None
Client --> Service --> Dependency
```

2. Open State (Broken)

Failure threshold exceeded.

- Requests blocked immediately.
- No calls sent downstream.

```None
Client --> Service --X--> Dependency
```

3. Half-Open State (Recovery Test)

After cooldown period:

- Allow limited test requests.
- If success → Closed
- If fail → Open again

Flow Example

Threshold:

- 50 failures in 1 minute

Sequence:

1. Payment service failing.
2. Errors exceed threshold.
3. Circuit opens for 30 sec.
4. After 30 sec, allow 5 test requests.
5. If successful, close circuit.

Key Parameters

Parameter	Meaning
Failure threshold	Number/% failures to trip
Timeout window	Time interval checked

Cooldown time	How long circuit stays open
Half-open request count	Test requests allowed

Benefits

1. Prevent Cascading Failure

One failing service doesn't crash all services.

2. Faster User Response

Fail fast instead of waiting timeout.

3. Protect Resources

Avoid thread exhaustion / CPU waste.

4. Auto Recovery

System retries later automatically.

Common Fallback Strategies

When circuit open:

- Return cached data
- Show “temporarily unavailable”
- Queue request
- Use default response
- Use backup provider

Example Use Cases

E-commerce

Checkout → Payment gateway

Food Delivery

App → Maps API

Video Platform

Home page → Recommendation service

Banking

Transaction → Fraud detection API

Popular Tools

Resilience4j
Hystrix
Netflix

Tradeoffs

Advantage	Tradeoff
Prevent overload	More complexity
Faster failure	Hard tuning

Auto recovery	False triggering possible

Challenges

1. Wrong Threshold

Too low → trips too often
Too high → reacts too late

2. Testing Complexity

Load tests may bypass true system limits.

3. Hidden Dependency Issues

If fallback masks outage too long.

Circuit Breaker vs Retry

Retry	Circuit Breaker
Try again after failure	Stop trying after many failures
Good for transient issues	Good for persistent failures

Best used together.

Retry + Circuit Breaker Pattern

1. Retry 2 times
2. If repeated failures continue
3. Open circuit

Interview Answer Structure

When asked fault tolerance:

> I'd add circuit breakers around external dependencies like payment, email, search, and recommendation services. If error rate exceeds threshold, requests fail fast and fallback response is served. Half-open mode tests recovery.

Short Interview Answer

> Circuit breaker prevents repeated calls to failing downstream services by opening the circuit after too many errors, then testing recovery later using half-open state.

One-Line Memory Trick

> Retry for temporary failures, circuit breaker for repeated failures.